

Rendimiento (3 apartados)

1. Speedup

Recupera el problema de la Sesión 1, y si no lo hiciste entonces, **calcula ahora el Tex (tiempo de ejecución) en cada procesador (mono y multiciclo) y el Speedup del más rápido respecto al más lento**, al ejecutar la siguiente mezcla de instrucciones (*instruction mix*) simplificada a partir del *benchmark* de empujados MediaBench:

- 40 % lw, 10% sw, 25% beq (86 % de ellos tomados, 14% no tomados), 25% ALUops
- Para que no pierdas tiempo, hemos calculado un Tc de **131 ps para la monociclo**, y de **68 ps para la multiciclo**. Si quieres hacerlo tú “para entrenar”, hemos asumido los siguientes tiempos en ps:
 - Registros tsup = td = 2; BR: tsup =5, Td=20
 - Muxes (todos): camino datos 4, camino contrl 2
 - ALU: 20; Alu salto (monociclo): 10; ALU PC (monociclo): 4
 - Extensión de signo = desplazador (salto) = 4
 - Multiciclo: MD/MI-> Tsup = 0; Td=60
 - Monociclo: Td MI = Td MD = 40; Tsup MD = 10; and (Z) = 2;
 - Considerad la UC fuera del camino crítico
- **Calcula la frecuencia en GHz** de cada procesador

2. Amdhal

En la multiciclo anterior, tal como lo vimos en clase, lw y sw leen y escriben en memoria respectivamente en 1 ciclo. Supón que esa ruta consigue ese acceso en 1 ciclo gracias a que hemos optimizado la memoria con una estructura interna que no nos importa ahora mismo. En la ruta inicial no optimizada, con una DRAM, cuando llegan a la etapa M a cada lw y a cada sw les cuesta leer / escribir 100 ciclos. Las instrucciones continúan leyéndose en 1 ciclo.

Calcula de nuevo el Tex y el Sup.

- Con el mix anterior, **¿Qué % del Tex supone la ejecución de lw y sw?** Es decir, **cuál es la F**, la fracción de tiempo sobre la que influye nuestra memoria mejorada de 1 ciclo?
- **¿Cuál es el speedup parcial (el factor ‘a’)** por el que dividimos esa F?
- **Repite ahora el cálculo con este nuevo *instruction mix*:** 5% lw, 5% sw, 25% beq (86 tomados, 14% no tomados) y 75% ALUops
 - ¿Cuál es el **speedup parcial**, F/a , con F igual que antes (= fracción de tiempo de ejecución de lw+sw)?
 - ¿Cuál es el **speedup global**?
 - ¿Qué **conclusiones** sacas?

3. GIPS, Gflops, EPI

Para evaluar el rendimiento de un micro ejecutamos un programa de benchmark y medimos lo siguiente:

- $I = 100 \times 10^6$ millones (I -> total de instrucciones de lenguaje máquina ejecutadas)
- 80 millones de instrucciones en coma flotante, precisión simple (*Flops*).
- 200 W de potencia consumida
- Tiempo de CPU: 6250 ms

Calcula:

- GIPS (*Giga-instrucciones por segundo*, $1G \text{ decimal} = 10^9$)
- Gflops (Giga-instrucciones en coma flotante por segundo)
- EPI (*Energy per instruction*, en **nJ por instrucción**)

Responde:

- Qué métricas de las anteriores me sirven para comparar el rendimiento si repito la ejecución del mismo programa, compilado con las mismas opciones, sobre
 - Otro micro con ISA diferente?
 - Otro micro con la misma ISA?

Nota orientativa:

- Para los Gflops solo consideramos las operaciones de lenguaje máquina en coma flotante. Loads o stores de registros de coma flotante no se cuentan. Las instrucciones de lenguaje máquina especiales como por ejemplo saxpy ($a * c + b$) o daxpy (lo mismo pero en doble precisión) no cuentan.
- Ejemplo (de las slides del T03). En un bucle como el siguiente

```
for( i=0; i<VECSIZE; i++){a(i) = b(i) * c(i) + 3.0; }
```

en donde a, b y c son vectores de VECSIZE de floats en precisión simple, si se compila para que el compilador sólo utilice instrucciones convencionales, solo hay dos instrucciones en coma flotante: un producto (e.g. `fmul`) y una suma (e.g. `fadd`). Si el bucle se ejecuta 1 millón de veces, se habrán ejecutado 2 millones de instrucciones en coma flotante en ese bucle